1) Explain how the programming language constructs like declaration, assignment statements, boolean expression and procedure call can be translated into an intermediate form

Ans: In compiler design, a source pgm written in a high-level language is translated into an intermediate representation (IR) before generating machine code. The intermediate form simplies the compilation process and allows optimization. One common intermediate representation is three-address code.

Programming constructs such as assignments, declarations, Boolean expressions, and procedure calls can be translated into intermediate code using translation schemes attached to grammar productions.

Assignment statements

Suppose that the context in which an assignment appears is given by the following grammar.

$P \rightarrow MD$

$M \rightarrow \varepsilon$

$D \rightarrow D; D \mid id : T \mid proc\ id;\ ND; S$

$N \rightarrow \varepsilon$

Non terminal P becomes the new start symbol when these productions are added to those in the translation scheme shown below

Translation scheme to produce three-address code for assignments:

$S \rightarrow id := E$ { P: lookup(id.name)
if P ≠ nil then
emit (P ':=' E.place)
else error }

$E \rightarrow E_1 + E_2$ { E.place := newtemp;
emit (E.place ':=' E_1.place + E_2 place) }

$E \rightarrow E_1 * E_2$ { E.place = newtemp;
emit (E.place ':=' E_1.place '*' E_2.place) }

$E \rightarrow E_1$ { E.place := newtemp;
emit(E.place ':=' uminus ' E_1. place) }

$E \rightarrow (E_1)$ { E.place = E_1.place }

$E \rightarrow id$ { P: lookup (id.name);
if P ≠ nil then
E.place := P
else error }

Declarations in a Procedure

The syntax of languages such as C, Pascal and Fortran, allows all the declarations in a single procedure to be processed as a group. A global variable, say offset can keep track of the next available relative address.

Translation scheme:

→ Nonterminal P generates a sequence of declarations of the form id:T.

→ Before the first declaration is considered, offset is set to 0. As each new name is seen that name is entered in the symbol table with offset equal to correct value of offset, and offset is incremented by the width of the data object denoted by that name.

→ The procedure enter (name, type, offset) creates a symbol table entry for name given its type and relative address offset in its data area.

Computing the types and relative address of declared names:

$P \rightarrow D$	{offset := 0}
$D \rightarrow D ; D$	
$D \rightarrow id : T$	{ enter(id.name, T.type, offset): offset := offset + T.width }
$T \rightarrow integer$	{ T.type := integer; T.width := 4 }
$T \rightarrow real$	{ T.type := real; T.width := 8 }
$T \rightarrow array[num]$ of T_1	{ T.type := array(num.val, T type): T.width := num.val x + T_1 width }
$T \rightarrow T_1$	{ T.type = pointer (T type); T.width := 4 }

Boolean Expression

Boolean expressions have two primary purposes. They are used to compute logical values, but more often they are used as conditional expressions in statements that alter the flow of control such as if then else, or while do statements.

Boolean expressions are composed of boolean operators and, or and not applied to elements that boolean variables or relational expressions - <, >, <>, != It is the format E_1 relop E_2 where E_1 and E_2 are arithmetic expressions

$E \rightarrow E$ or $E \mid E$ and $E \mid (E)$ id relop | id | true | false

Translation scheme using a numerical representation for boolean

$E \rightarrow E_1$ or E_2 {E.place := newtemp;
emit (E place := 'E_1 place' or E_2 place)}

$E \rightarrow E_1$ and E_2 {E place := newtemp;
emit (E place := E_1 place 'and' E_2 place)}

$E \rightarrow$ not E_1 {E place := newtemp;
emit (E place := not E_1 place)}

$E \rightarrow (E_1)$

$E \rightarrow$ id relop id_2 {E.place = E_1 place}
{E place := newtemp;
emit (if id_1 place relop op id_2 place 'go to' nextstat +3).
emit (goto next stat +2);
emit (E place := '1');

$E \rightarrow$ true {E place = newtemp
emit (E place := '1');
{E place := '1'}
emit (E place := '1')

$E \rightarrow$ false {E place := newtemp:
emit (E place ':= '0)

Procedure calls

The procedure is such an important and frequently used programming construct that is imperative for a compiler to generate good code for procedure calls and returns. The run time routines and handle procedure argument passing, calls and returns are part of the run time support package.

let us consider a grammar for a simple procedure call statement

1) $S \rightarrow$ call id (Elist)
2) Elist $\rightarrow$ Elist , E
3) Elist $\rightarrow$ E

Calling sequences:

The translation for a call includes a calling sequence, a sequence of action taken on entry to and exit from each procedure. The failing are action that take place in a calling sequence:

→ when a procedure call occurs, space must be allocated for the activation record of the called procedure.

→ the arguments of the called procedure must be evaluated and made available to the called procedure in a known place.

→ Environment pointers must be established to enable the called procedures to access data in enclosing ~~brackets~~ blocks.

→ The state of the calling procedure must be saved so it can resume execution after the call.

Eg: 1) $S \rightarrow$ call id (Elist)

{ for each item p on queue do
emit ('param' p);
emit ('call' id.place) }

2) Elist $\rightarrow$ Elist E

{ append E.place to the end of queue }

3) Elist $\rightarrow$ E

{ initialise queue to contain only E.place }